

Egy egyszerű és hatékony algoritmus egy minimális feszítőfa helyettesítő éleinek megtalálására

David A. Bader, Paul Burkhardt

<http://jgaa.info/> vol. 26, no. 4, pp. 577-588 (2022)

Nagy Bence

2025. december 1.

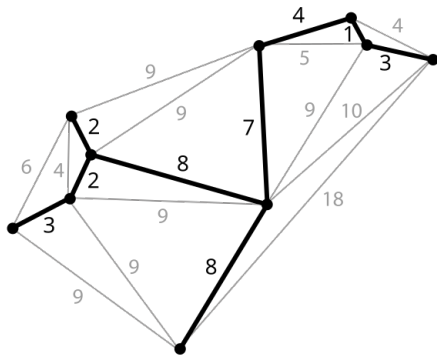
Áttekintés: minimális feszítőfa, csereél

Definíció:

Legyen $G = (V, E)$ egy irányítatlan, súlyozott gráf, ahol $n = |V|$ és $m = |E|$, és minden $e \in E$ élhez tartozik egy $w(e)$ súly. Egy minimális feszítőfa $T = MST(G)$ a gráf olyan $n - 1$ élből álló részhalmaza, amely összeköti az összes csúcsot, és az élek összsúlya minimális.

Definíció:

A csereél az a legkisebb súlyú él, amely újra összeköti a minimális feszítőt egy minimális feszítőfabeli él törlése után.



Kép forrás: Wikipédia

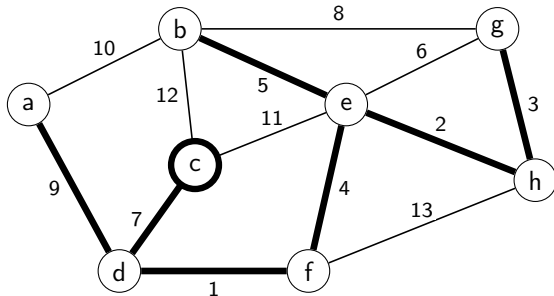
- Adottak:
 - A gráf minimális feszítőfája
 - A nem feszítőfabeli élek RENDEZVE
- Keressük:
 - Csereéleket minden minimális feszítőfabeli élhez.

- Az algoritmus képes megtalálni minden csereélet, ha a nem min feszítőfabeli élek élsúly szerint növekvő sorrendben rendezve megvannak határozva:
 - **Idő- és tárbonyolultság:** $\mathcal{O}(m + n)$
(m : élek száma, n : csúcsok száma)
- A cikk előtt lehetett tudni, hogy ez elméletileg lehetséges, de ez az első algoritmikus megoldás erre lineáris időkomplexitással.

- 1975: Spira és Pan: MST egy éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1978: Chin és Houck: MST minden éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1979: Tarjan: $\mathcal{O}(m\alpha(m, n))$
 - α : Ackermann függvény inverze: nagyon-nagyon lassan nő a bemenethez képest.
 - pl: $\alpha(7) = 2$, $\alpha(61) = 3$, $\alpha(2^{2^{2^{2^{2^2}}}} - 3) = 4$
 - Majdnem lineáris megoldás, de mégsem az.
- 2022: Bader és Burkhardt: $\mathcal{O}(m + n)$

Az algoritmus alapötlete 1.

- Adott T és a nem-fa élek $E \setminus E_T$ súly szerint növekvő sorban.
- Figyeljük meg, hogy mindegyik $m - n + 1$ darab él $E \setminus E_T$ -ben egy fundamentális kört indukál T éleivel.
- Then for any MST edge there is a subset of cycles containing that edge, and the cycle induced by the lightest non-MST edge is the replacement for it

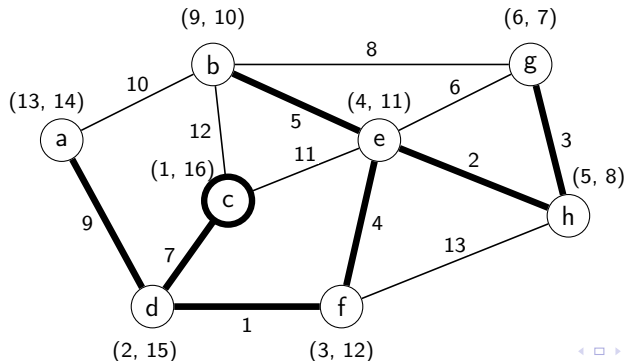


Itt: az (e,g) él az (e,h) és a (g,h) élek minimális költségű csereéle.

Az algoritmus alapötlete 2.

- 1 Assign the parent and the first and last visited numbers to each vertex according to depth-first search over T
- 2 Traverse the fundamental cycle induced by each non-tree edge in order of ascending weight
- 3 Compress paths to skip edges already assigned replacements

We use the disjoint sets for fast path compression based on the Gabow-Tarjan static union tree method.



Definíció:

A diszjunkt halmaz adatstruktúra diszjunkt halmazok gyűjteményét tartja fenn:

$\mathcal{S} = \{S_1, S_2, \dots, S_k\}$. Minden halmazt egy **képviselőjével** azonosítjuk, amely tagja a halmaznak.

Műveleteket a diszjunkt halmaz adatstruktúrán:

- **makeSet(v)**: létrehoz egy új egy elemű halmazt: $\{v\}$, amelynek v lesz a képviselője.
- **find(v)**: visszaadja annak a diszjunkt halmaznak a képviselőjét, amelyben v van.
- **link(u, v)**: Létrehoz egy új halmazt, ami u és v halmazának az uniója.

Gabow és Tarjan algoritmusa a diszjunkt halmazok egy speciális esetére

- Speciális eset: a teljes uniófa (Union Tree) ismert az algoritmus indulásakor.
- Esetünkben az eredeti MST lesz a Union-Tree.
- Az algoritmus **m darab unió és find műveletet** hajt végre, **n darab elemen**: $\mathcal{O}(m + n)$ futási időben, $\mathcal{O}(n)$ tárhelykomplexitással
- Itt: $\text{link}(v)$: v halmazát és $P[v]$ halmazát összeuniózza és $P[v]$ képviselője lesz az új halmaz képviselője. ($P[v]$: v szülője a Union-Tree-ben)

Az algoritmus pseudokódja

- 1: Root the MST T at arbitrary vertex v_r and store parents in P .
- 2: $P[v_r] \leftarrow v_r$ ▷ root's parent points to root
- 3: Run DFS on T , setting $IN[v]$ and $OUT[v]$ to the counter value when v is first and last visited.
- 4: **for all** vertices $v \in V$ **do**
- 5: $\text{MAKESET}(v)$ ▷ Initialize the disjoint sets
- 6: **for all** edges $e \in E_T$ **do**
- 7: $R_e \leftarrow \emptyset$ ▷ Initialize the replacement edges
- 8: **for** $k \leftarrow 1$ **to** $m - n + 1$ **do**
- 9: $(v_i, v_j) \leftarrow e_k$ ▷ Scan the $m - n + 1$ sorted non-MST edges
- 10: $\text{PATHLABEL}(v_i, v_j, (v_i, v_j))$
- 11: $\text{PATHLABEL}(v_j, v_i, (v_i, v_j))$

Az algoritmus pszeudokódja

```
1: procedure PATHLABEL( $s, t, e$ )
2: if  $IN[s] < IN[t] < OUT[s]$  then
3:   return
4: if  $IN[t] < IN[s] < OUT[t]$  then
5:    $PLAN \leftarrow ANCESTOR$ ;  $k_1 \leftarrow IN[t]$ ;  $k_2 \leftarrow IN[s]$ 
6: else
7:   if  $IN[s] < IN[t]$  then
8:      $PLAN \leftarrow LEFT$ ;  $k_1 \leftarrow OUT[s]$ ;  $k_2 \leftarrow IN[t]$ 
9:   else
10:     $PLAN \leftarrow RIGHT$ ;  $k_1 \leftarrow OUT[t]$ ;  $k_2 \leftarrow IN[s]$ 
11:  $v \leftarrow s$ 
12: while  $k_1 < k_2$  do
13:   if  $FIND(v) = v$  then
14:      $R(v, P[v]) \leftarrow e$ 
15:      $LINK(v)$ 
16:    $v \leftarrow FIND(v)$ 
17: switch  $PLAN$  do
18:   case  $ANCESTOR$ :  $k_2 \leftarrow IN[v]$ 
19:   case  $LEFT$ :  $k_1 \leftarrow OUT[v]$ 
20:   case  $RIGHT$ :  $k_2 \leftarrow IN[v]$ 
```

▷ s is ancestor of t

▷ t is ancestor of s

▷ s is left of t

▷ s is right of t

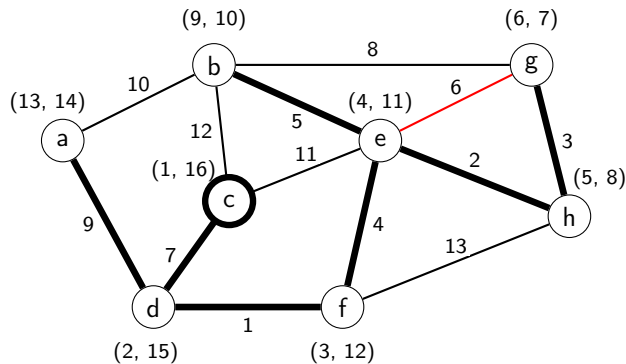
▷ Detecting when below $LCA(s, t)$

▷ If true, set replacement edge for $(v, P[v])$

▷ Set the replacement edge

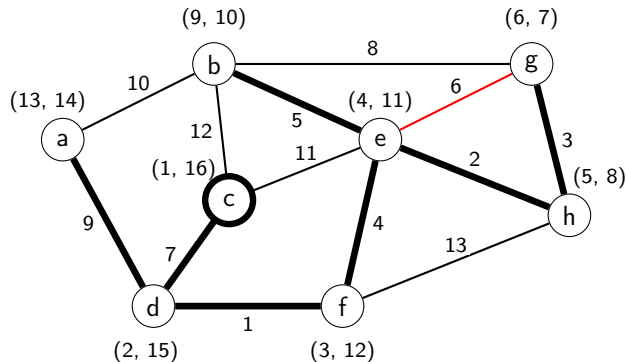
▷ Union the disjoint sets of v and $P[v]$

Példa lefutás - 1. él



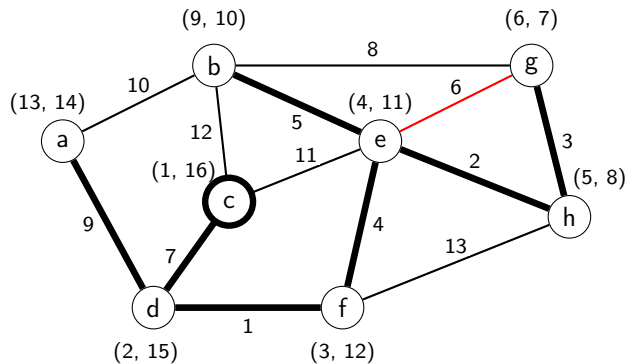
- 1 Vertex e is the ancestor of g, then we get the ancestor (ANCESTOR) plan with $k_1 = \text{IN}[e]$, $k_2 = \text{IN}[g]$ and thus $k_1 < k_2$.
- 2 The disjoint sets are linked so g's disjoint set gets h's label. This compresses the subpath (g,h)

Példa lefutás - 1. él (folytatás)



- 1 The next vertex is h since it is the parent of g and then k2 is updated to IN[h].
- 2 Continuing the traversal with h (line 12), again line 14 assigns (g,e) to (h,e) where e is the parent of h.
- 3 The disjoint sets are linked (line 15) so h's disjoint set gets e's label. This compresses the subpath (g h e).

Példa lefutás - 1. él (folytatás)



- 1 Now the next vertex is e so k_2 gets $IN[e]$ making it equal to k_1 , thus ending the while loop.
- 2 The oppositely-oriented edge (e,g) input at line 34 is not processed because e is the ancestor of g and we have already followed the path from descendent to ancestor

Tétel:

Adott egy irányítatlan, súlyozott $G = (V, E)$ gráf minimális feszítőfája, és a nem feszítőfabeli élek súly szerint rendezve, akkor az algoritmus $\mathcal{O}(m + n)$ időben és $\mathcal{O}(m + n)$ helyigény mellett megkeresni G minimális feszítőfájának összes minimális költségű csereélét.

Bizonyítás:

Legyen T a G gráf minimális feszítőfája.

- 1 A P szülő tömb inicializálása: $\mathcal{O}(n)$
- 2 Mivel T -ben $n - 1$ él van, ezért a DFS lefuttatása T -n az IN és OUT értékek inicializálásához: $\mathcal{O}(n)$.
- 3 Csereélek inicializálása minden MST élhez: $\mathcal{O}(n)$ idő.
- 4 $m - n + 1 = \mathcal{O}(m)$ nem-MST él olvasás van, növekvő sorban súly szerint: $\mathcal{O}(m)$ időkomplexitás.

Bizonyítás: (folyt.):

- **Definíció:** Az az él, amelynek eltávolítása a legnagyobb növekedést okozza az MST (minimális feszítőfa) súlyában.
- **Az algoritmus hozzájárulása:**
 - A csereélek meghatározása után a legfontosabb él kiválasztható $\mathcal{O}(n)$ időben. Csak meg kell keresni, hogy melyik MST-beli él súlyának a különbsége legnagyobb a csereélének súlyával.
 - Így: a legfontosabb él kiszámítható $\mathcal{O}(m + n)$ időben, ha adott az eredeti MST és a nem MST élek rendezve.

- Képesek vagyunk egy minimális feszítőfa minden élének a csereélét megtalálni $\mathcal{O}(m + n)$ időben, ha a nem MST éleket növekvő sorrendben kapjuk meg.
- Ehhez Gabow és Tarjan speciális diszjunkt halmaz adatszerkezetét használjuk fel.



Cormen, Thomas H and Leiserson, Charles E and Rivest, Ronald L and Stein, Clifford "Introduction to algorithms", *MIT press* (2022)

Köszönöm a figyelmet!